REPORT
PY-NMAP

TECHNOLOGIES: This project utilizes various technologies to organize and streamline its development. These are described below, along with their respective functions:

- PYTHON 3: Python was chosen as the programming language due to its ease of use, user-friendly syntax, and extensive community support.

- OOP (Object-Oriented Programming): The use of Object-Oriented Programming (OOP) facilitates future project enhancements and improves organization, allowing the developer to easily navigate and modify specific parts of the codebase.

- FUNCTIONS: Functions were employed to avoid code repetition, making the codebase less redundant and more elegant.

- CONDITIONAL STRUCTURES: Conditional structures are crucial to this project, as they enable the user to make decisions regarding which commands to execute.

- REPETITION STRUCTURES (LOOPS): Loops are used for various simple tasks, but primarily to support error

handling; if a user enters an invalid input, the program allows them to try again.

- ERRORS AND EXCEPTIONS: These ensure that the program does not crash if the user enters an invalid or disallowed input.

- NETWORK LIBRARIES: Network libraries serve as the core component that brings the program to life. Technologies such as sockets allow the user to retrieve information—such as IP addresses, ports, and host details—without the developer having to manually implement every aspect of the process.

- AI: Artificial Intelligence also played an important role in this project, assisting the developer—who is currently learning about network libraries and networking concepts—throughout the development process.

DEVELOPMENT: The Py-Nmap project was created to simulate the Nmap security tool, enabling an operator to perform TCP port scans. Since the project was based on Nmap, several challenges arose—such as working with network libraries like sockets—where the use of artificial intelligence proved crucial. Maintaining an organized codebase to prevent it from becoming a complete mess, while also ensuring the absence of errors, was another key challenge.
The initial concept was simply to create a basic scanner capable of checking for open ports, such as 80 or 443. Over time, the scope expanded to incorporate features similar to

those found in Nmap—such as latency measurement and service identification (banner grabbing)—ultimately resulting in a more complex project that closely resembles the Nmap tool itself.